TryHackMe Writeup: Splunk - Exploring SPL Room:

Category: Cyber Defense / SOC Level 1 / SIEM
Difficulty: Medium
Completion Status: 100% Completed
Target Index: `index=windowslogs`

Author: James Thiongo

Executive Summary

The **Splunk: Exploring SPL** room introduces the fundamentals of Splunk's **Search Processing Language (SPL)**, the query language used to search, filter, analyze, and visualize machine-generated data. Throughout this lab, I explored how Security Operations Center (SOC) analysts leverage SPL to investigate Windows Event Logs and Sysmon logs, identify suspicious activity, summarize large datasets, and perform behavioral analysis.

The exercises covered essential SPL concepts including searching indexed data, applying Boolean operators, filtering events, restructuring search results, generating statistical summaries, enriching log data with geolocation information, and detecting anomalous user behavior using statistical methods.

Lab Environment

Component	Description
Platform	TryHackMe
Tool	Splunk Enterprise
Data Source	Windows Event Logs & Sysmon Logs
Query Language	Search Processing Language (SPL)
Primary Index	`index=windowslogs`

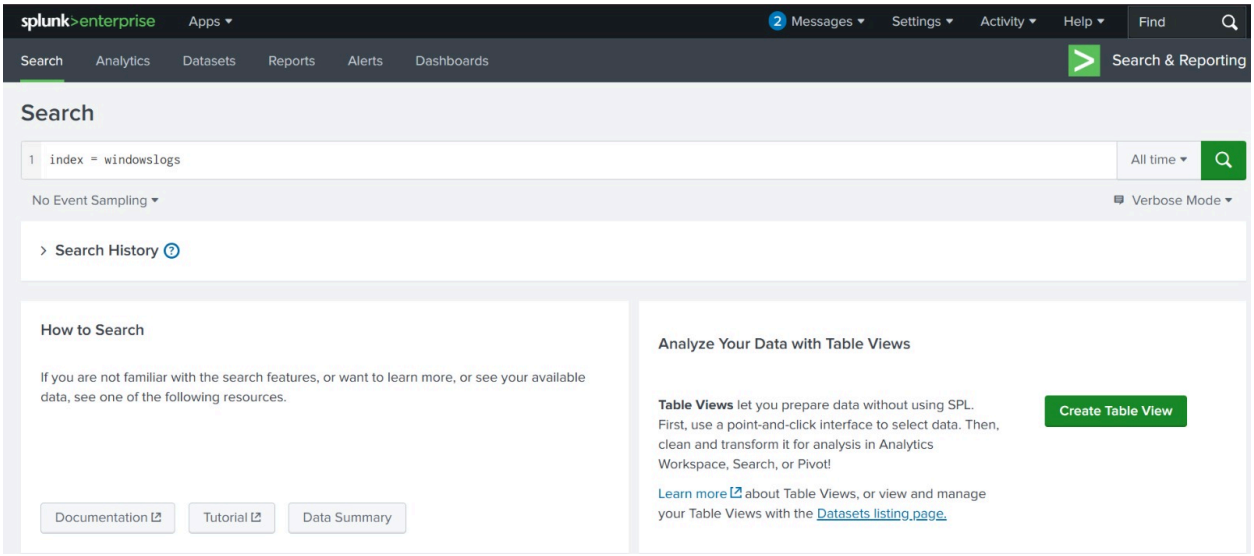
Task 1 — Introduction

Objective

Understand the purpose of the room and launch the provided Splunk instance.

Actions Performed

- Started the virtual machine.
- Opened the Splunk Search & Reporting interface.
- Familiarized myself with the investigation environment.



Task 2: Search & Reporting

Overview

This task introduced the **Search & Reporting** application in Splunk. I learned how to perform baseline searches, adjust the search timeframe using the **Time Picker**, and use the **Fields Sidebar** to quickly identify important values within indexed Windows event logs.

Running a Baseline Search

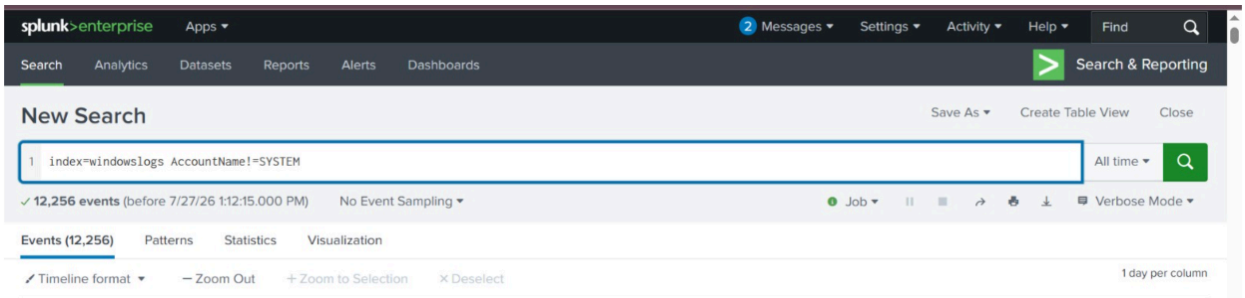
To begin the investigation, I executed a search against the `windowslogs` index and configured the **Time Picker** to **All Time** to ensure the complete dataset was available for analysis.

SPL Search

```
index=windowslogs
```

Result

- **Total Events: 12,256**



Analyst Note

Running a baseline search confirms that the expected data has been indexed and is available for investigation. Verifying the **Time Picker** at the start of an investigation helps prevent relevant historical events from being unintentionally excluded.

Exploring the Fields Sidebar

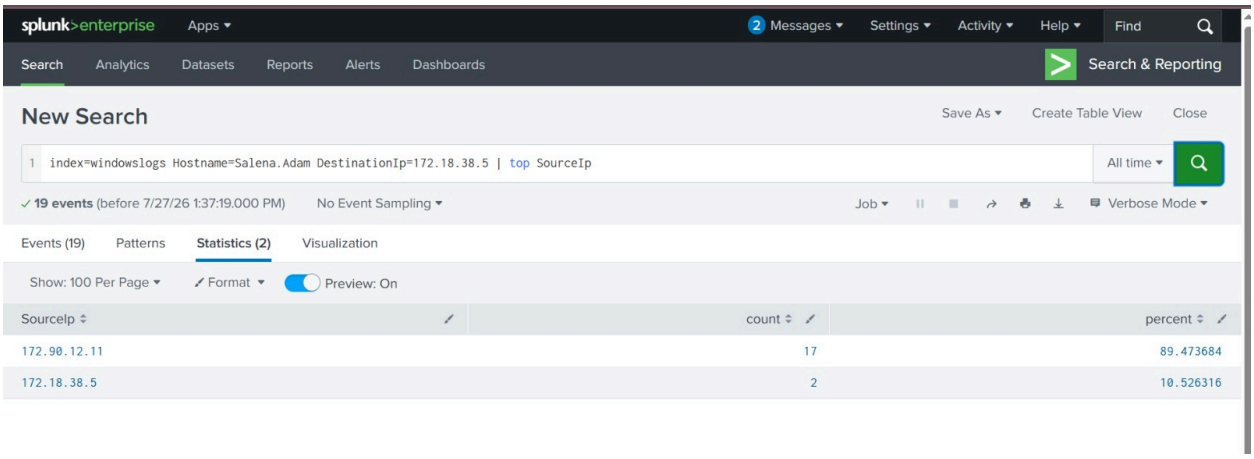
The **Fields Sidebar** provides a quick overview of extracted field values without requiring additional SPL commands. To identify the most active source IP address, I examined the **SourceIp** field and confirmed the result using the `top` command.

SPL Search

```
index=windowslogs
| top SourceIp
```

Result

- **Most Frequent Source IP:** `172.90.12.11`



Analyst Note

The Fields Sidebar helps analysts quickly identify frequently occurring users, hosts, IP addresses, and other field values. This provides an efficient starting point for identifying notable activity before performing deeper analysis.

Filtering Events by Time

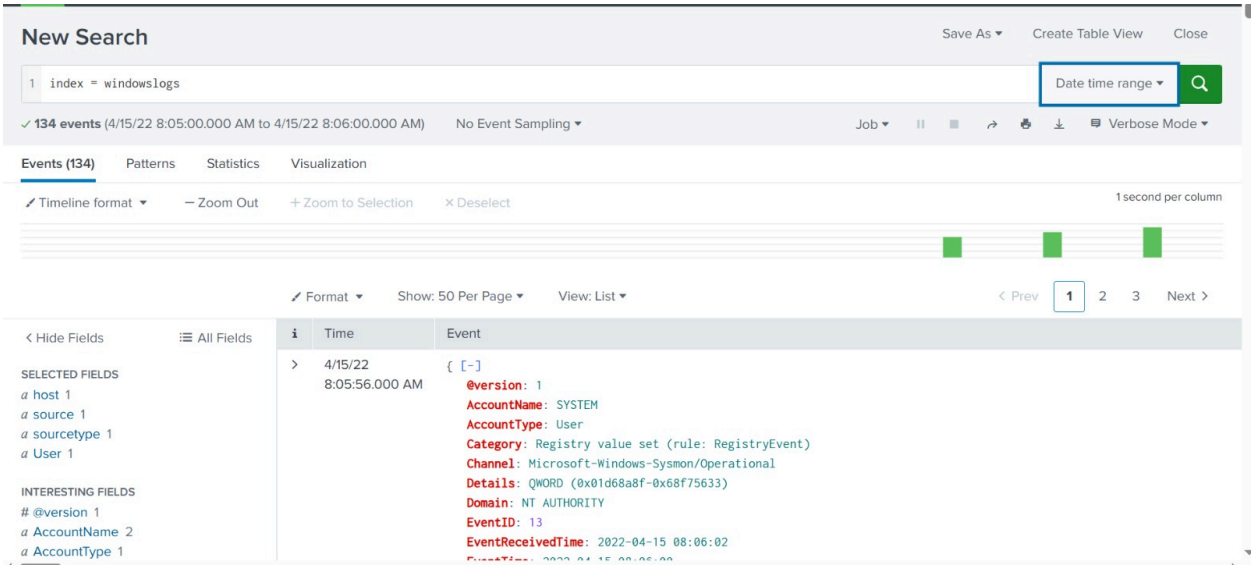
Using the **Time Picker**, I restricted the search to a one-minute interval to isolate events generated during a specific period.

SPL Search

```
index=windowslogs
```

Result

- **Matching Events: 134**



Analyst Note

Time-based filtering is a fundamental investigative technique that narrows the scope of an analysis, improves search performance, and helps analysts reconstruct event timelines during incident response.

Task 3: Search Operators

Overview

This task introduced the use of **Search Processing Language (SPL) search operators** to efficiently locate and filter events within the `windowslogs` index. By combining field-value searches, Boolean operators, wildcard matching, and operator precedence, I learned how to narrow large datasets into meaningful results that support security investigations.

Field-Value Searches

Field-value searches are the foundation of SPL. They allow analysts to retrieve only events containing specific field values, significantly reducing the amount of data that needs to be reviewed.

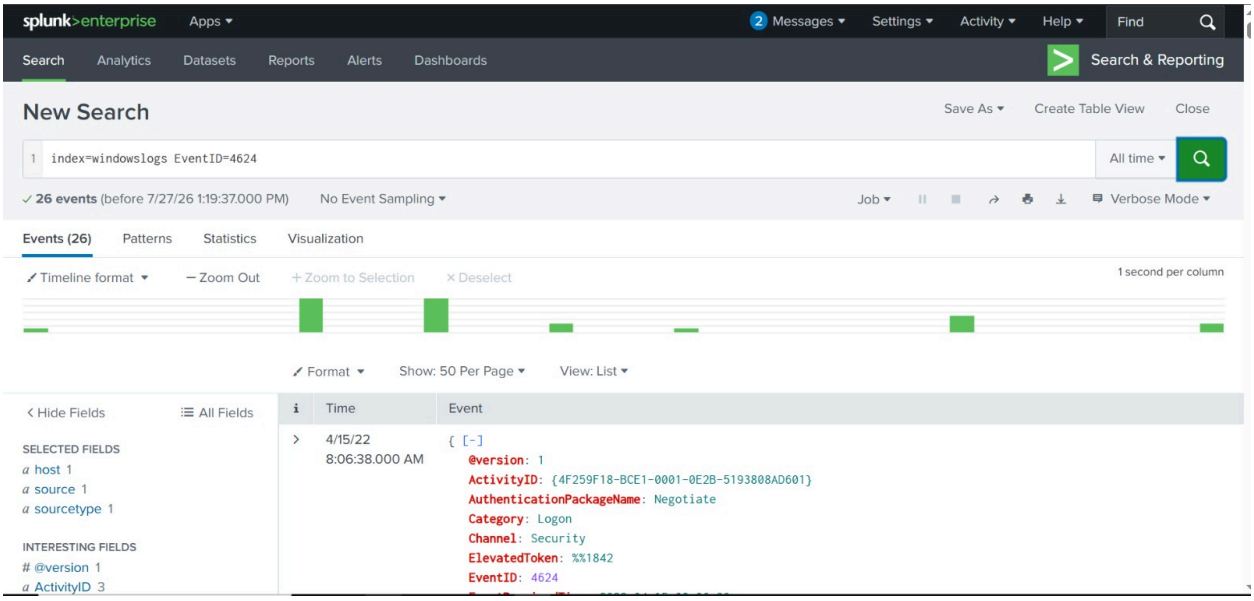
For example, to retrieve all successful Windows logon events, I searched for Event ID **4624**.

SPL Search

```
index=windowslogs EventID=4624
```

Result

- **Total Events: 26**



Analyst Note

Windows **Event ID 4624** represents a successful user logon. During investigations, analysts frequently examine these events to establish user activity timelines, detect unusual authentication behavior, and identify privileged account usage.

Combining Multiple Search Conditions

SPL allows multiple field-value pairs to be combined within a single search. Unless otherwise specified, Splunk interprets these conditions using an implicit **AND** operator.

SPL Search

```
index=windowslogs DestinationIp="172.18.39.6" DestinationPort=135
```

or

```
index=windowslogs DestinationIp="172.18.39.6" DestinationPort=135
| stats count
```

Result

- **Matching Events: 4**

Analyst Note

Combining multiple search conditions allows analysts to isolate highly specific activity. In this example, only events matching both the destination IP address and RPC port **135** were returned. This technique is particularly useful when investigating lateral movement or remote administration activity.

Frequency Analysis Using the `top` Command

The `top` command identifies the most frequently occurring values within a selected field.

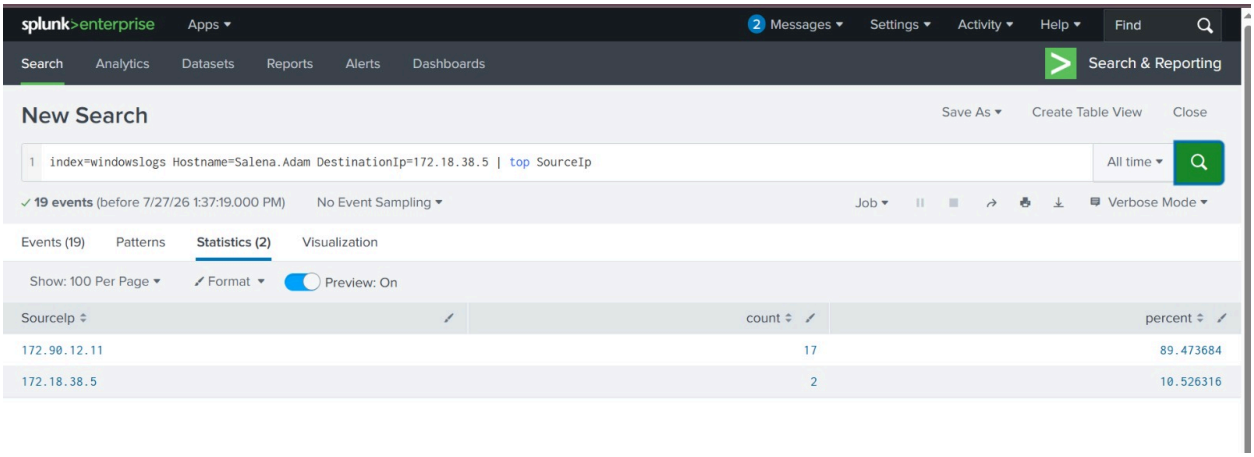
Using a filtered dataset, I determined which source IP communicated most frequently with a particular destination host.

SPL Search

```
index=windowslogs Hostname=Salena.Adam DestinationIp=172.18.38.5
| top SourceIp
```

Result

- **Most Frequent Source IP: 172.18.39.100**



Analyst Note

The `top` command automatically calculates event frequency, percentage distribution, and sorts the results from highest to lowest. This makes it useful for identifying dominant users, hosts, IP addresses, or processes during threat hunting.

Wildcard Searches

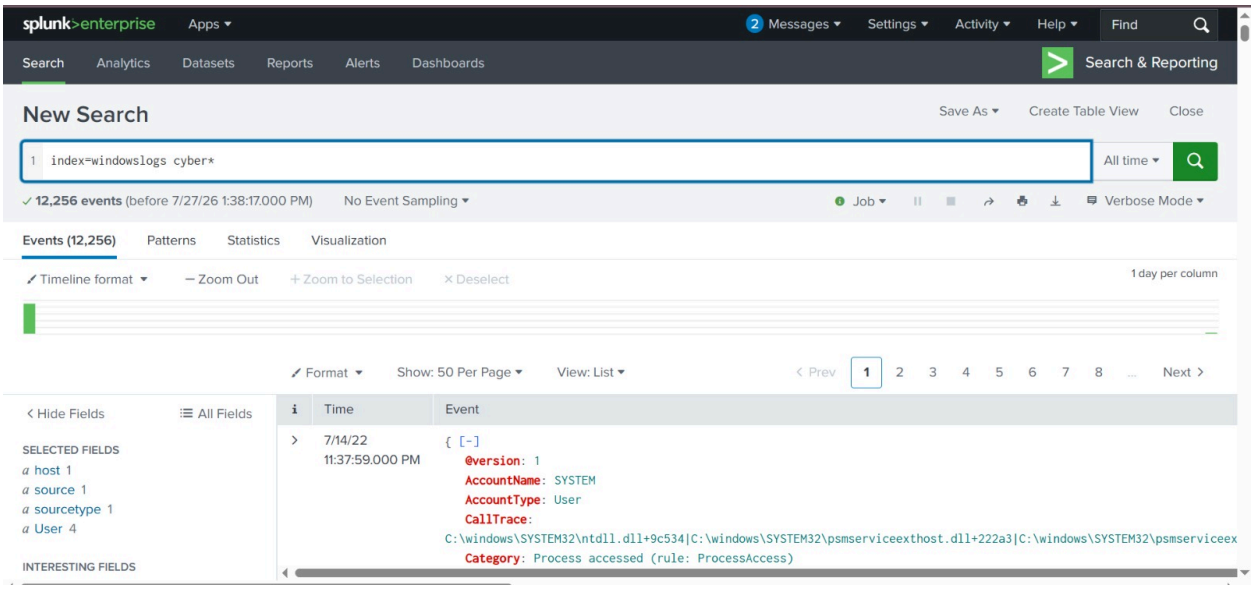
Wildcards allow analysts to search for partial keywords when the complete value is unknown.

SPL Search

```
index=windowslogs cyber*
```

Result

- **Matching Events:** 12256



Analyst Note

Wildcard searches are valuable when hunting for unknown variations of filenames, processes, commands, or user accounts. However, they should be used carefully, as broad wildcard searches can increase search time by requiring Splunk to evaluate a larger number of indexed events.

Boolean Logic and Operator Precedence

Splunk evaluates Boolean expressions according to a defined order of precedence:

1. Quotation Marks (`" "`)
2. Parentheses (`()`)
3. `NOT`
4. `AND`
5. `OR`

Since `OR` has the lowest precedence, parentheses should be used whenever multiple Boolean operators are combined.

Example

```
index=windowslogs (EventID=4624 OR EventID=4625) Hostname=Salena.Adam
```

This query returns successful and failed logon events only for the specified host.

Analyst Note

Understanding operator precedence is essential for constructing accurate searches. Incorrectly grouped conditions can produce unintended results and potentially cause analysts to overlook relevant security events.

Task 4: Filtering Results

Overview

This task introduced SPL commands used to filter and refine search results before performing further analysis. By reducing unnecessary data, analysts can improve query performance, simplify investigations, and focus on the most relevant events. The task explored commands such as `fields`, `sort`, `stats`, and `regex` to efficiently organize and filter Windows event logs.

Selecting Relevant Fields with `fields`

The `fields` command allows analysts to control which fields are retained in the search pipeline. By displaying only the information relevant to an investigation, searches become easier to interpret while reducing unnecessary processing of unused fields.

To identify the highest **Source Process ID**, I limited the output to the `Domain`, `SourceProcessId`, and `TargetProcessId` fields before sorting the results.

SPL Search

```
index=windowslogs
| fields Domain, SourceProcessId, TargetProcessId
| sort - SourceProcessId
```

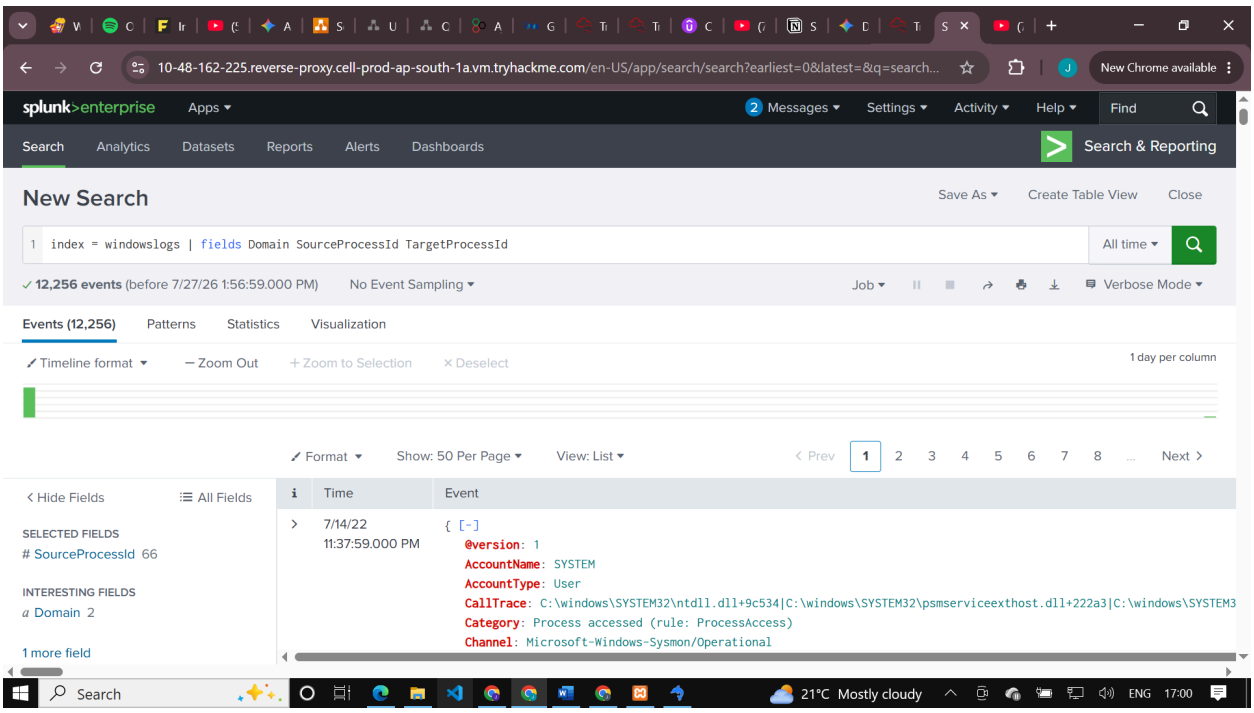
Alternative Search

The same result can also be obtained by calculating the maximum value directly.

```
index=windowslogs
| fields Domain, SourceProcessId, TargetProcessId
| stats max(SourceProcessId)
```

Result

- **Highest SourceProcessId:** 9496



Analyst Note

The `fields` command removes unnecessary metadata from the search pipeline, making results easier to read and improving search efficiency. Combining `fields`

with `sort` allows analysts to quickly identify extreme values, while the `stats max()` function provides a more efficient method when only the highest value is required.

Pattern Matching with `regex`

The `regex` command enables filtering based on **Perl Compatible Regular Expressions (PCRE)**, allowing analysts to search for values that match specific text patterns rather than exact strings.

In this investigation, I filtered registry paths ending with the word **"Manager"** and identified the most frequently occurring value.

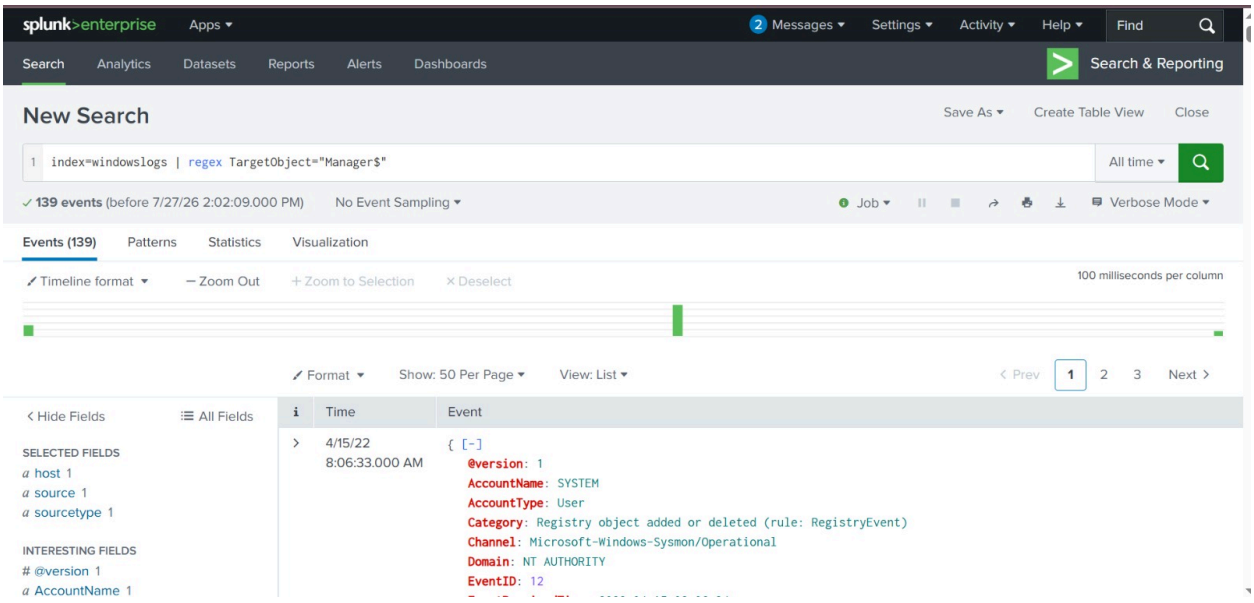
SPL Search

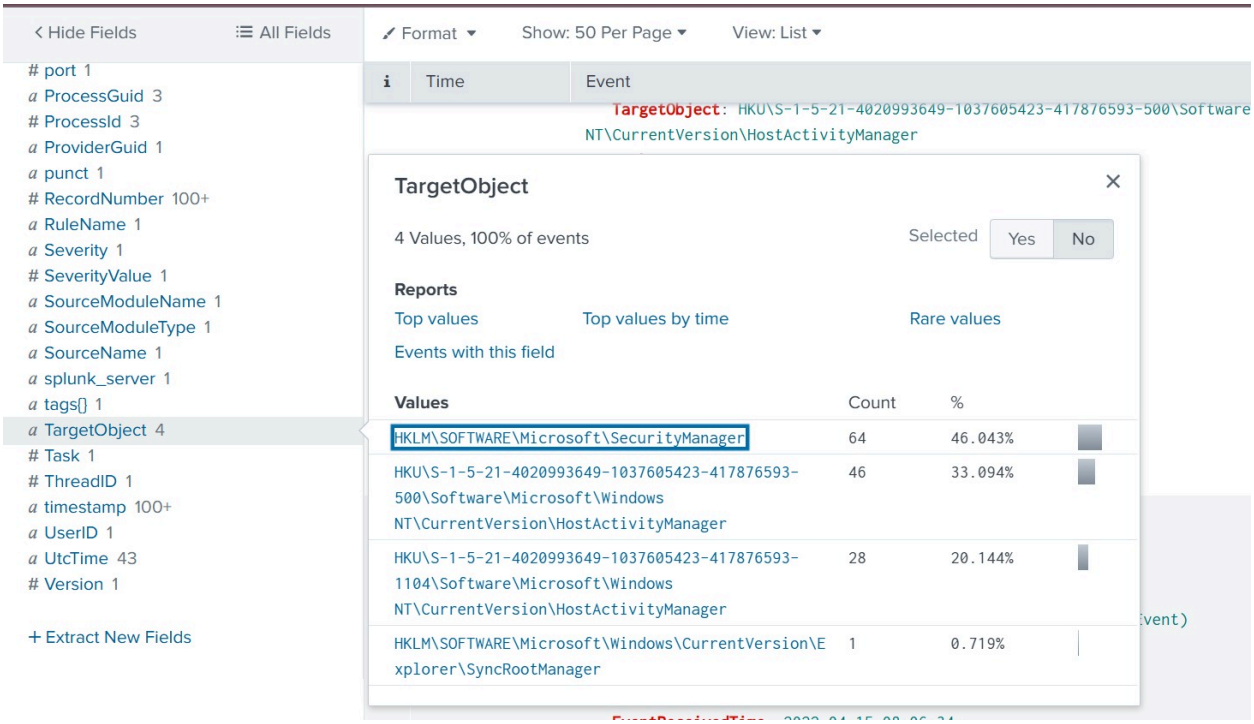
```
index=windowslogs
| regex TargetObject="Manager$"
| top TargetObject
```

Result

Most Frequent TargetObject:

```
HKLM\SOFTWARE\Microsoft\SecurityManager
```





Analyst Note

Regular expressions provide a powerful method for identifying patterns within log data. Instead of searching for exact values, analysts can match file extensions, registry paths, command-line arguments, or process names using flexible expressions.

For example:

Pattern	Purpose
`\.exe$`	Match executable files ending with `.exe`
`Manager$`	Match strings ending with "Manager"
`^HKLM`	Match registry paths beginning with `HKLM`
`.*powershell.*`	Match any string containing "powershell"

Task 5: Structuring Results

Overview

This task focused on structuring raw log data into organized, readable formats suitable for security investigations and reporting. Instead of reviewing unstructured event logs, I learned how to present relevant information using commands such as `table`, `reverse`, and timeline analysis techniques. These commands help analysts reconstruct attack sequences, improve report readability, and quickly identify key evidence during an investigation.

Presenting Data with the `table` Command

Raw log events often contain dozens of fields, many of which are irrelevant to a specific investigation. The `table` command allows analysts to display only the fields required for analysis, making search results significantly easier to interpret.

To view authentication-related information, I created a table containing the **Event ID**, **Account Name**, and **Account Type** fields.

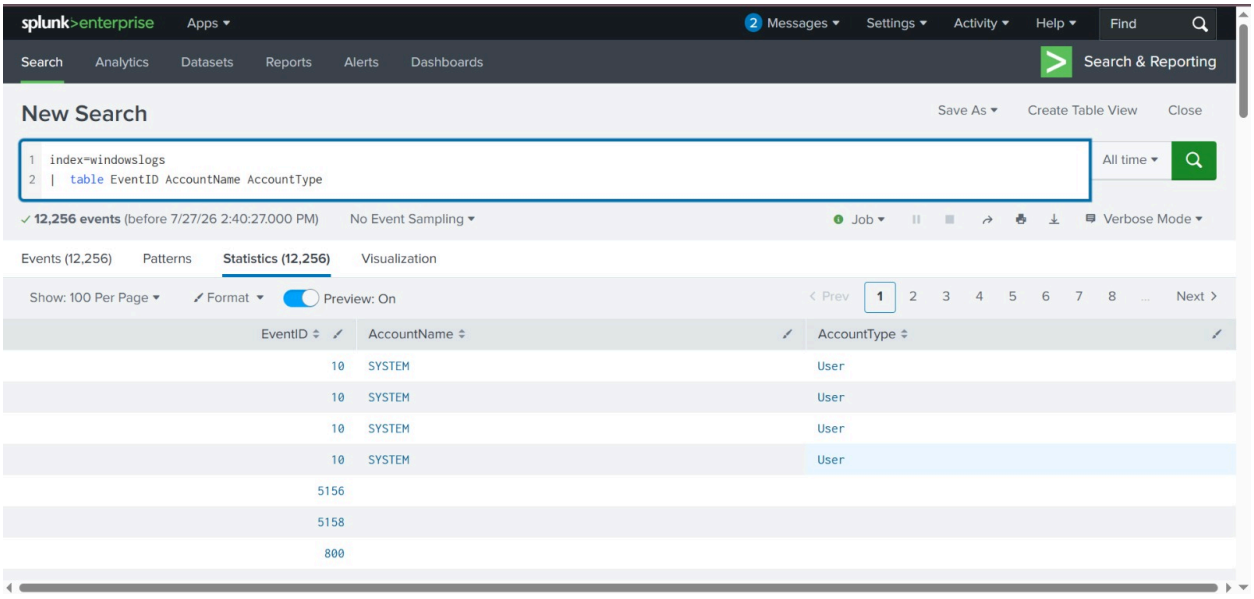
SPL Search

```
index=windowslogs
| table EventID AccountName AccountType
```

Result

- **First Account Name Displayed: SYSTEM**

Search results formatted using the `table` command.



Analyst Note

The `table` command restructures raw search results into a clean, column-based format, making it easier to identify important information without scrolling through unnecessary metadata. By default, Splunk displays events in **reverse chronological order**, meaning the most recent events appear first.

Reconstructing Event Timelines with `reverse`

Understanding the order in which events occurred is essential during incident response. Since Splunk returns the newest events first, the `reverse` command can be used to display events in chronological order.

Appending `reverse` to the previous search produced the earliest recorded event at the top of the results.

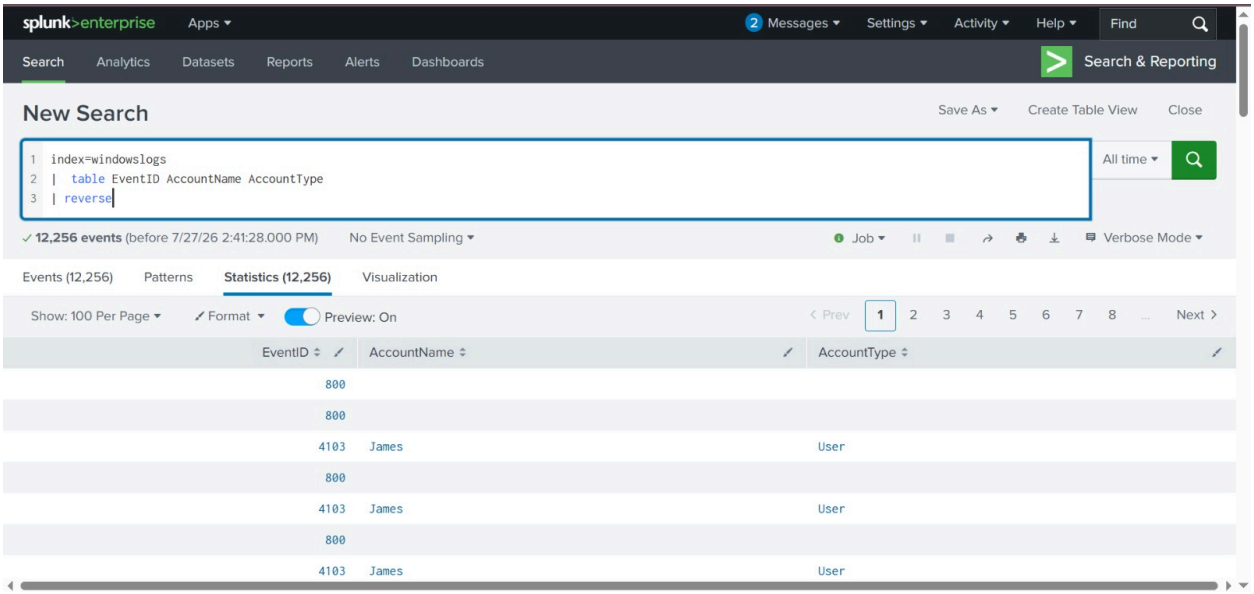
SPL Search

```
index=windowslogs
| table EventID AccountName AccountType
```

```
| reverse
```

Result

- **First Event ID:** 800



Analyst Note

The `reverse` command is particularly useful when reconstructing an attack timeline. Viewing events from oldest to newest enables analysts to trace an incident from the initial point of compromise through subsequent actions such as privilege escalation, lateral movement, and persistence.

Timeline Analysis of Process Creation Events

Process creation logs provide valuable insight into commands executed on a system. By filtering **Sysmon Event ID 1** and displaying command-line arguments, I reconstructed the sequence of executed processes to identify evidence of account creation activity.

SPL Search

```
index=windowslogs EventID=1
| table _time ParentProcessId ProcessId ParentCommandLine CommandLine
| reverse
```

Result

During the timeline review, the following password assigned to the user **A1berto** was identified:

- **Password:** paw0rd1

Analyst Note

Sysmon Event ID 1 records detailed process creation information, including the command-line arguments used to launch each process. This data is invaluable during investigations because it can reveal administrative actions, scripting activity, malware execution, or attacker behavior.

In this example, reviewing the command-line history exposed the password used during user account creation, illustrating how process telemetry can provide critical forensic evidence.

> **Note:** While plaintext passwords appeared in this controlled training environment, exposing passwords on the command line is generally considered an insecure practice in production environments because command-line arguments may be visible to other users or captured in logs.

Task 6: Transforming Commands

Overview

This task focused on **transforming commands**, which summarize, enrich, and aggregate raw log data into meaningful information for security analysis. I used commands such as `top`, `stats`, `iplocation`, and `lookup` to identify common processes, enrich IP addresses with geographic information, and correlate events with external risk data.

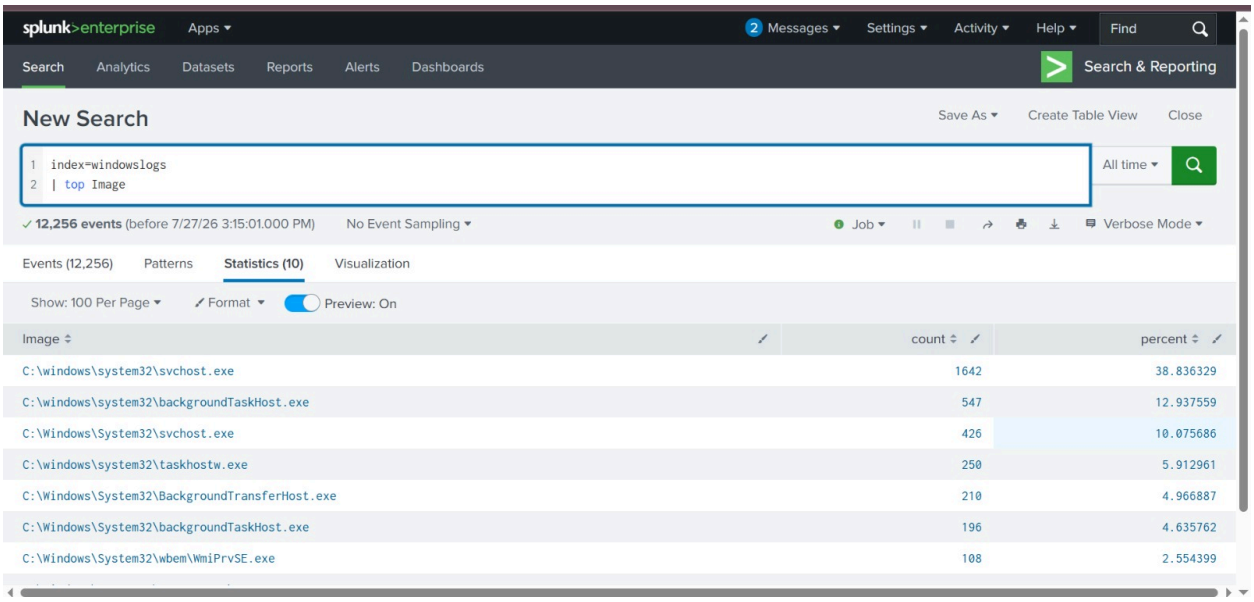
Identifying Common Processes with `top`

The `top` command displays the most frequently occurring values in a field. I used it to identify the process image that appeared most often in the dataset.

SPL Search

```
index=windowslogs
| top Image
```

Result: `C:\Windows\System32\svchost.exe`



Analyst Note: `top` quickly highlights the most common values within a dataset, helping analysts identify normal activity as well as unusual spikes that may require investigation.

Enriching IP Addresses with `iplocation`

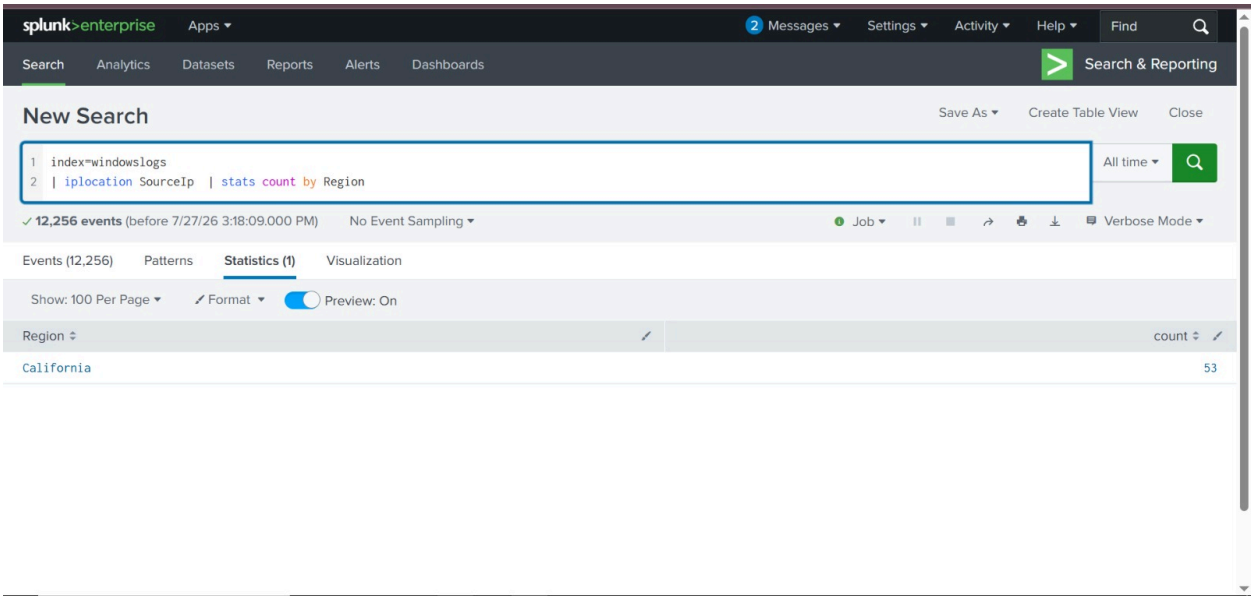
The `iplocation` command adds geographic information to IP addresses, providing additional context during investigations.

SPL Search

```
index=windowslogs
| iplocation SourceIp
```

```
| stats count by Region
```

Result: Carlifonia



Analyst Note: Geolocation data helps analysts investigate login activity, identify unexpected connection origins, and support threat hunting by adding context to network events.

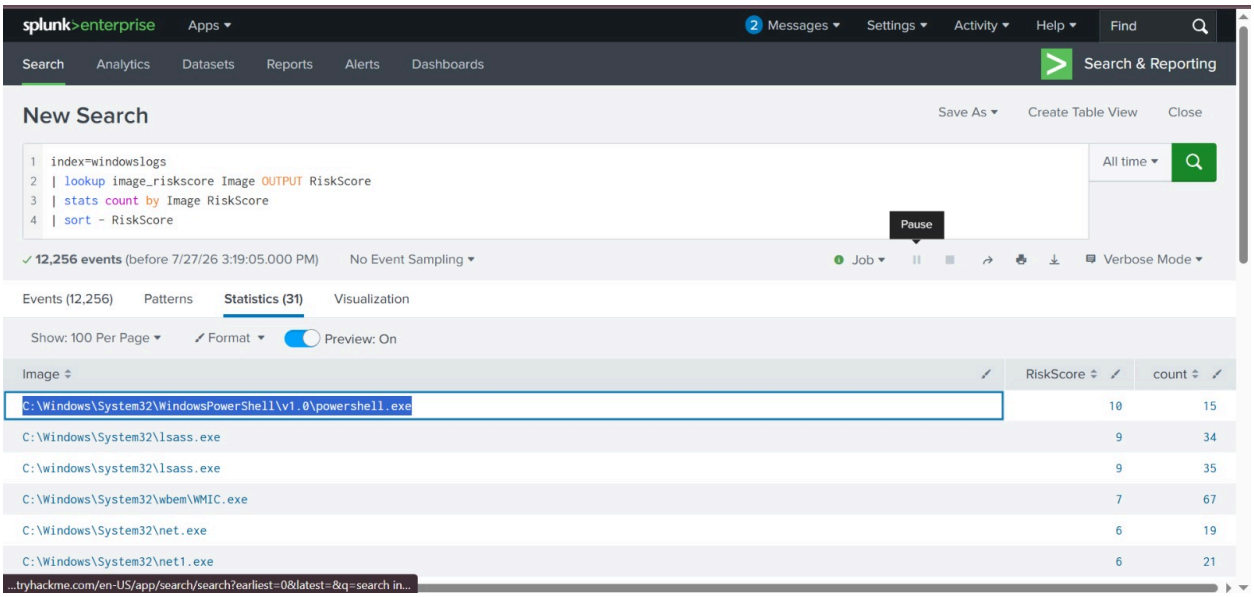
Correlating Events with `lookup`

The `lookup` command enriches events using external reference tables. In this exercise, I matched process images with predefined risk scores.

SPL Search

```
index=windowslogs
| lookup image_riskscore Image OUTPUT RiskScore
| stats count by Image RiskScore
| sort - RiskScore
```

Result:



Analyst Note: Lookups provide additional context by integrating external data such as threat intelligence, asset information, or risk scores into search results.

Task 7: Anomaly Detection

Overview

This task explored behavioral anomaly detection in Splunk by identifying unusual VPN login locations and abnormal login times using statistical analysis. These techniques help analysts establish user baselines and detect activities that may indicate compromised accounts or unauthorized access.

Detecting Geographic Login Anomalies

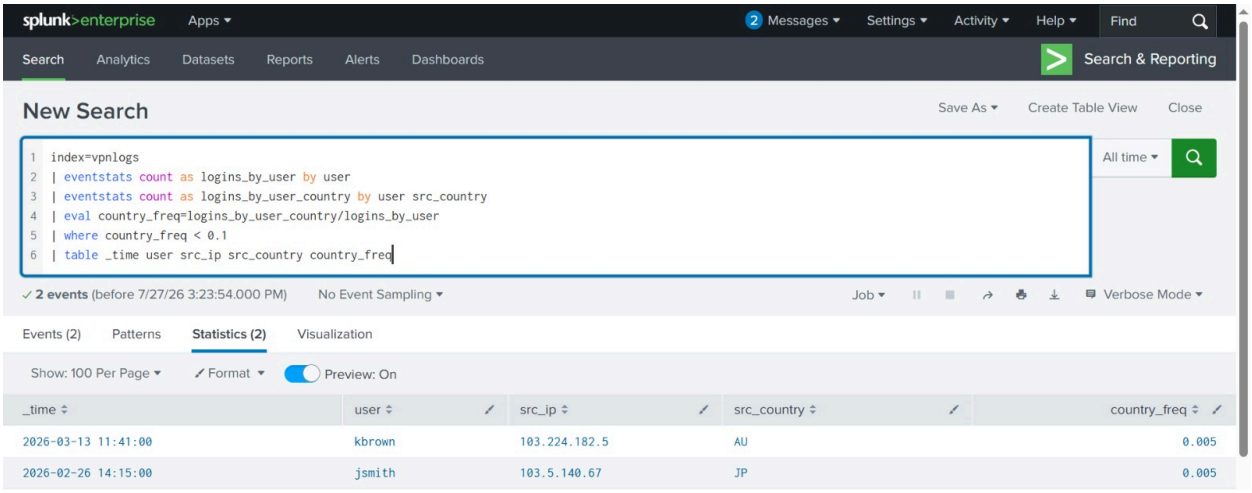
To identify unusual VPN login locations, I used a query that calculated how frequently each user logged in from different countries. Logins originating from countries with a very low historical frequency were flagged as potential anomalies.

SPL Search

```
index=vpnlogs
| eventstats count as logins_by_user by user
| eventstats count as logins_by_user_country by user src_country
| eval country_freq=logins_by_user_country/logins_by_user
| where country_freq < 0.1
| table _time user src_ip src_country country_freq
```

Result

- **Outlier User:** `jsmith`
- **Anomalous Country:** `JP`



Analyst Note

Geographic baselines help identify unusual login locations. Although an unexpected country is not definitive evidence of compromise, it is a useful indicator that should be investigated alongside other authentication and network events.

Detecting Time-Based Login Anomalies

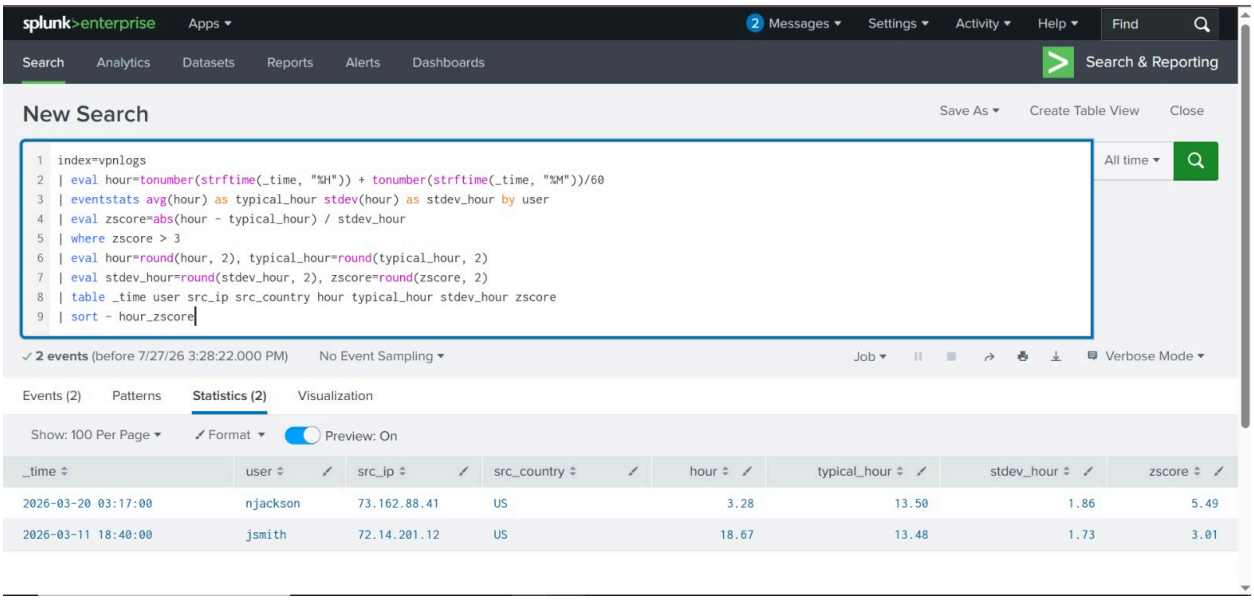
To identify unusual login times, I calculated each user's average login hour and standard deviation, then used a **Z-score** to detect events that significantly deviated from their normal behavior.

SPL Search

```
index=vpnlogs
| eval hour=tonumber(strftime(_time,"%H")) + tonumber(strftime(_time,"%M"))/60
| eventstats avg(hour) as typical_hour stdev(hour) as stdev_hour by user
| eval zscore=abs(hour-typical_hour)/stdev_hour
| where zscore > 3
| table _time user src_ip src_country hour typical_hour stdev_hour zscore
| sort - zscore
```

Result

- **Anomalous User:** njackson
- **Suspicious Login Time: 3:17 AM**



Analyst Note

Z-score analysis identifies login events that significantly deviate from a user's normal behavior, helping analysts detect suspicious activity while reducing false positives.

Conclusion

The **Splunk: Exploring SPL** room provided hands-on experience with Splunk's Search Processing Language (SPL) and demonstrated how it can be used to investigate Windows and Sysmon logs. Throughout the lab, I practiced searching indexed data, filtering and structuring events, enriching logs with contextual information, and applying statistical techniques to detect anomalous behavior.

These exercises reinforced core SOC skills, including log analysis, event correlation, timeline reconstruction, and behavioral analytics. Overall, the room strengthened my understanding of how SPL supports security monitoring, threat hunting, and incident response workflows.